

3-5 · Pandas와 DuckDB를 이용한 하이브리드 가공 · 실습 1.5H

정제 파이프라인 구현

기준선을 재고, 함수로 정제하고, 두 번 돌린다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① DuckDB 로 orders.csv 의 품질 기준선(before)을 젠다
- ② clean_orders.py — 추출은 DuckDB, 정제는 Pandas 로 쓴다
- ③ 두 번 돌려 equals() 가 True 인지(역등성) 확인한다
- ④ validate() 를 만들고 일부러 깨뜨려 실제로 잡는지 본다

결과물 orders_clean.parquet 와 clean_orders.py 가 다음 1H 리포트의 재료입니다

여섯 단계 — ~/data-eng/week3 한 폴더에서

3-1 의 orders.csv 가 없으면 make_orders.py 를 다시 실행합니다 — 시드가 고정돼 언제 만들어도 같은 원본

1. 기준선	2. clean()	3. 두 번 돌리기	4. 순서 실험	5.6. 검증·완성
DuckDB 로 before 측정	clean_orders.py 1015 → 1000 행	equals() True 세 번	중앙값 A·B 비교	validate() 종료 코드

원본의 품질 기준선 — DuckDB 한 문장

rows	count(*) → 1015
null_price	count(*) - count(price) → 30 이상
dup_orders	count(*) - count(DISTINCT order_id) → 15
category_values	count(DISTINCT category) → 4 보다 큼
오염 확인	GROUP BY category — '식품' 과 ' 식품 ' 이 따로

이 수치가 리포트의 before — 화면에선 구분이 안 되는 공백을 DB 는 다른 값으로 본다 (일관성 문제의 전형)

clean() 다섯 줄 — 한 줄씩 뜻

줄	뜻
str.strip()	category 공백 오염 제거
pd.to_datetime()	order_date 타입 복원
drop_duplicates()	order_id 기준, 첫 행 유지
fillna(median())	price 결측을 중앙값으로
price * quantity	amount 파생 컬럼

load_raw() 는 duckdb.sql("SELECT * FROM 'orders.csv']").df() — 대용량이면 여기서 필터·열 선택으로 줄인다

clean() 은 df.copy() 로 시작 — 원본을 건드리지 않겠다는 선언이고, 3-1 의 SettingWithCopyWarning 도 예방

돌리고, 두 번 돌리고 — 출력을 읽는다

1 python3 clean_orders.py
「1015 행 → 1000 행 정제 완료」 — 중복 15건 제거, orders_clean.parquet 생성

2 clean(load_raw()) 두 번
행 수 같은가 · 내용 같은가(equals) · 파일로도 같은가 — 세 줄 모두 True

3 하나라도 False 면
정제 규칙에 무작위성이나 순서 의존이 섞인 것 — t1·t2.parquet 는 rm 으로 정리

확인할 글자

True
세 번

순서를 바꾸면 중앙값이 달라지는가

- 1 순서 A — 중복 제거 후 중앙값**
`drop_duplicates(subset='order_id')` 뒤 `price.median()` → `med_a`
- 2 순서 B — 중복 있는 채로 중앙값**
`raw.copy()` 그대로 `price.median()` → `med_b`
- 3 같은가?**
이 데이터에선 같거나 아주 비슷할 수 있다 — 중복이 15건뿐이라 영향이 작다

기억할 것

**달라질 수
있다**

값이 같게 나왔다고 순서가 상관없는 것은 아니다 — 데이터가 바뀌면 달라진다. 실습 데이터에서 우연히 같았다고 리포트에 적어 둔다

검사기를 만들었으면 일부러 깨뜨린다

지금 데이터엔 없지만, 검사를 넣어 두면 나중에 원본이 바뀌었을 때 잡아 줍니다

1 validate()

음수 가격·수량
미래날짜·미지 분류

2 정제본 검사

네 항목 모두 0
= 유효성 통과

3 나쁜 값 주입

price -1000
category '알수없음'

4 다시 검사

price_le_0 1
unknown_cat 1

완성형 — 검증 실패면 종료 코드 1

VALID_CATEGORIES	['식품', '가전', '의류', '도서']
validate() 5항목	price_le_0 · quantity_le_0 · unknown_cat · dup_order_id · null_price
실패하면	print('검증 실패:', issues) → raise SystemExit(1)
통과하면	orders_clean.parquet 저장 · 「정제 완료 (검증 통과)」
확인	python3 clean_orders.py; echo "종료 코드: \$?" → 0

1-2 에서 배운 종료 코드가 여기서 쓰인다 — 자동화할 때 이 숫자가 성공 여부를 알려 준다

이 1.5시간이 끝날 때 확인할 것

- 1 baseline 에 rows 1015 · dup_orders 15 가 찍힌다
- 2 GROUP BY category 에 '식품' 과 ' 식품 ' 이 따로 나온다
- 3 '1015 행 → 1000 행 정제 완료' · orders_clean.parquet 생성
- 4 두 번 돌려 세 줄 모두 True
- 5 나쁜 값을 넣으면 price_le_0 1 · unknown_cat 1 로 잡힌다
- 6 최종본 실행 뒤 '검증 통과'와 종료 코드 0

3-5 · 실습 1.5H 마무리

다음은 데이터 품질 리포트 작성 (1H)

정제 전후를 같은 잣대로 재서 리포트를 만들고, DuckDB 로 교차 검증한 뒤 3주차 산출물로 제출합니다

✓ `quality_report.py` — before·after 프로파일

✓ 개선율 네 줄

✓ DuckDB 로 교차 검증

✓ 리포트 문서 작성 · 산출물 제출

가져갈 것

`orders_clean.parquet` 와 스크립트